



Fake Review Detection Using Machine Learning

Supervisor: Dr. Passent Elkafrawy

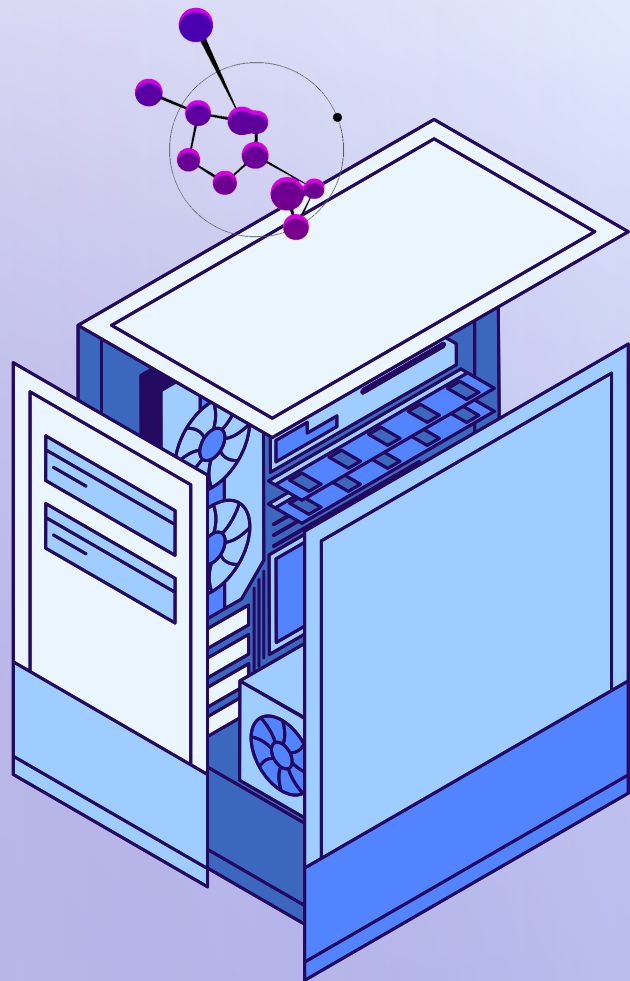
Course: Data Sciences CS2071

Semester: Fall 2025

Doneby: Rahaf Banat , Taif Alsulami , Ghazal Simbawah



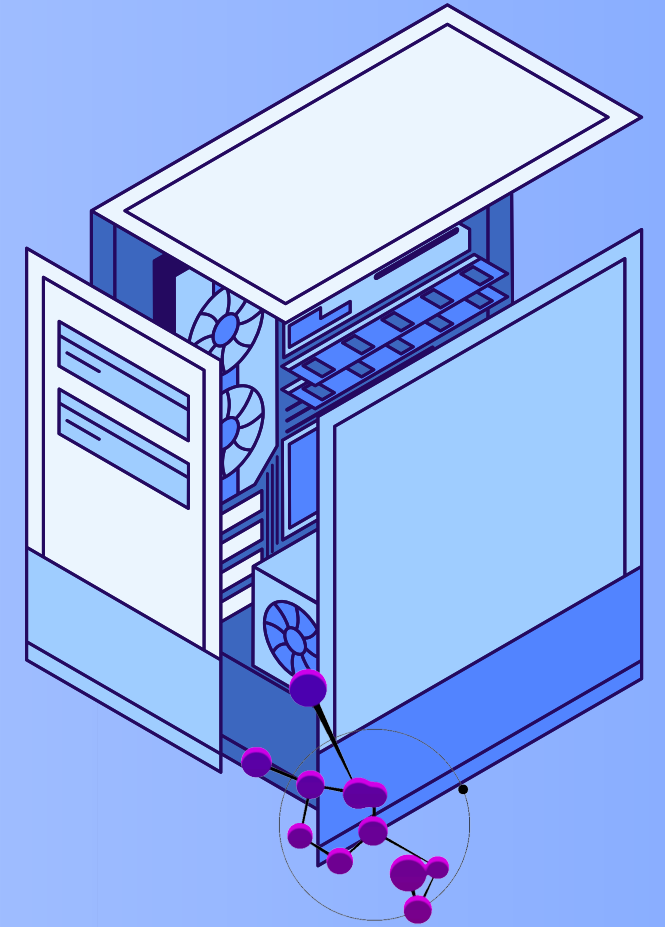
Introduction

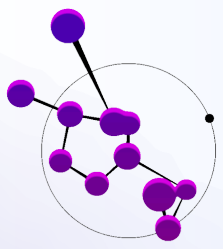


Fake reviews mislead customers, distort product ratings, and damage business credibility.

Because manual detection is difficult and unreliable, automated systems are needed.

Machine learning provides an effective way to identify suspicious reviews by learning patterns that distinguish fake from real feedback.





Introduction



Problem, Objectives & SDG Alignment

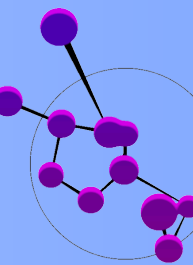
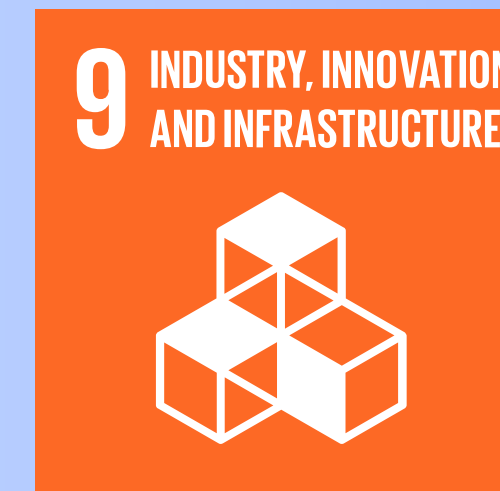
Problem Explanation

Fake reviews mislead customers, distort product ratings, and reduce trust in online shopping. Manual detection is unreliable, so an automated system is needed.

Project Objectives

- Build a clean, high-quality dataset
- Train and evaluate ML models to classify real vs fake reviews
- Deploy a simple interface for real-time prediction

SDGs:



Dataset overview

Dataset Name: Fake and Real Product Reviews Dataset

Dataset Description:

Amazon Customer Reviews is a real-world product review dataset collected over many years, widely used in NLP and ML research because it offers rich, diverse text data across many product categories.

Dataset Details:

Total Samples: 40,432

Columns: 4

Column Name	Description	Data Type
category	Product category of the reviewed item	object (string)
rating	Star rating given by the customer (1–5)	float64
label	Review classification (e.g., real, fake, trustworthy, suspicious)	object (string)
text_	Full customer review text	object (string)

Dataset overview

Sample Records from the Dataset:

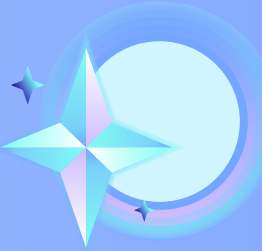
category	rating	label	text_
Home_and_Kitchen_5	5	CG	Love this! Well made, sturdy, and very comfortable...
Home_and_Kitchen_5	5	CG	Love it, a great upgrade from the original. I...
Home_and_Kitchen_5	5	CG	This pillow saved my back. I love the look and...
Home_and_Kitchen_5	1	CG	Missing information on how to use it, but it is...
Home_and_Kitchen_5	5	CG	Very nice set. Good quality. We have had the set...

Libraries & Data Cleaning steps

Library	Purpose
pandas	Data loading, cleaning, and DataFrame manipulation
numpy	Numerical operations and support functions
re	Regular expressions for removing URLs, numbers, emojis
string	Punctuation removal
unicodedata	Text normalization and standardization
matplotlib	Data visualization (plots, charts)
seaborn	Enhanced visualizations and statistical plots

- Label mapping (CG → real, OR → fake)
- Removing duplicates and contradictory reviews
- Lowercasing and Unicode normalization
- Removing URLs, emojis, punctuation, and numbers
- Removing extra spaces
- Filtering very short or very long reviews
- Creating word and character length features

Data Loading



```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="whitegrid")

# Load data
df = pd.read_csv("/content/fake_reviews_dataset.csv", on_bad_lines='skip', engine='python')
```

```
# Count missing values per column
missing = df.isna().sum().sort_values(ascending=False)
print("Missing values per column:")
print(missing)
```

Missing values per column:

```
category      0
rating        0
label         0
text_         0
label_text    0
label_bin     0
dtype: int64
```

```
df = df.drop_duplicates()
```

Data Cleaning



Label Cleaning

```
df["label_text"] = df["label"].map({"CG": "real", "OR": "fake"})  
df["label_bin"] = df["label_text"].map({"real": 0, "fake": 1})  
  
df = df.dropna(subset=["label_text", "label_bin"])  
print("After label mapping:", df.shape)
```

Add length features (word_len, char_len) and filter extremes

```
df["word_len"] = df["text_"].str.split().apply(len)  
df["char_len"] = df["text_"].str.len()  
  
df[["text_", "word_len", "char_len"]].head()
```


Filtering Extreme Review Lengths

```
df = df[df["word_len"] > 2]          # remove very short
    reviews
df = df[df["word_len"] < 500]        # remove extremely long
    reviews

print("After length filtering:", df.shape)
```

Text Normalization

```
import re
import string
import unicodedata

# Remove emojis
emoji_pattern = re.compile("[
    u\"\\U0001F600-\\U0001F64F\"
    u\"\\U0001F300-\\U0001F5FF\"
    u\"\\U0001F680-\\U0001F6FF\"
    u\"\\U0001F1E0-\\U0001F1FF\"
    \"]+", flags=re.UNICODE)

# Remove URLs
url_pattern = re.compile(r"http\\S+|www\\S+")

# Remove punctuation
punct_table = str.maketrans("", "", string.punctuation)

def final_clean(text):
    text = str(text).lower()
    text = unicodedata.normalize("NFKD", text)
    text = url_pattern.sub(" ", text)
    text = emoji_pattern.sub(" ", text)
    text = text.translate(punct_table)
    text = re.sub(r"\\d+", " ", text)
    text = re.sub(r"\\s+", " ", text).strip()
    return text
```

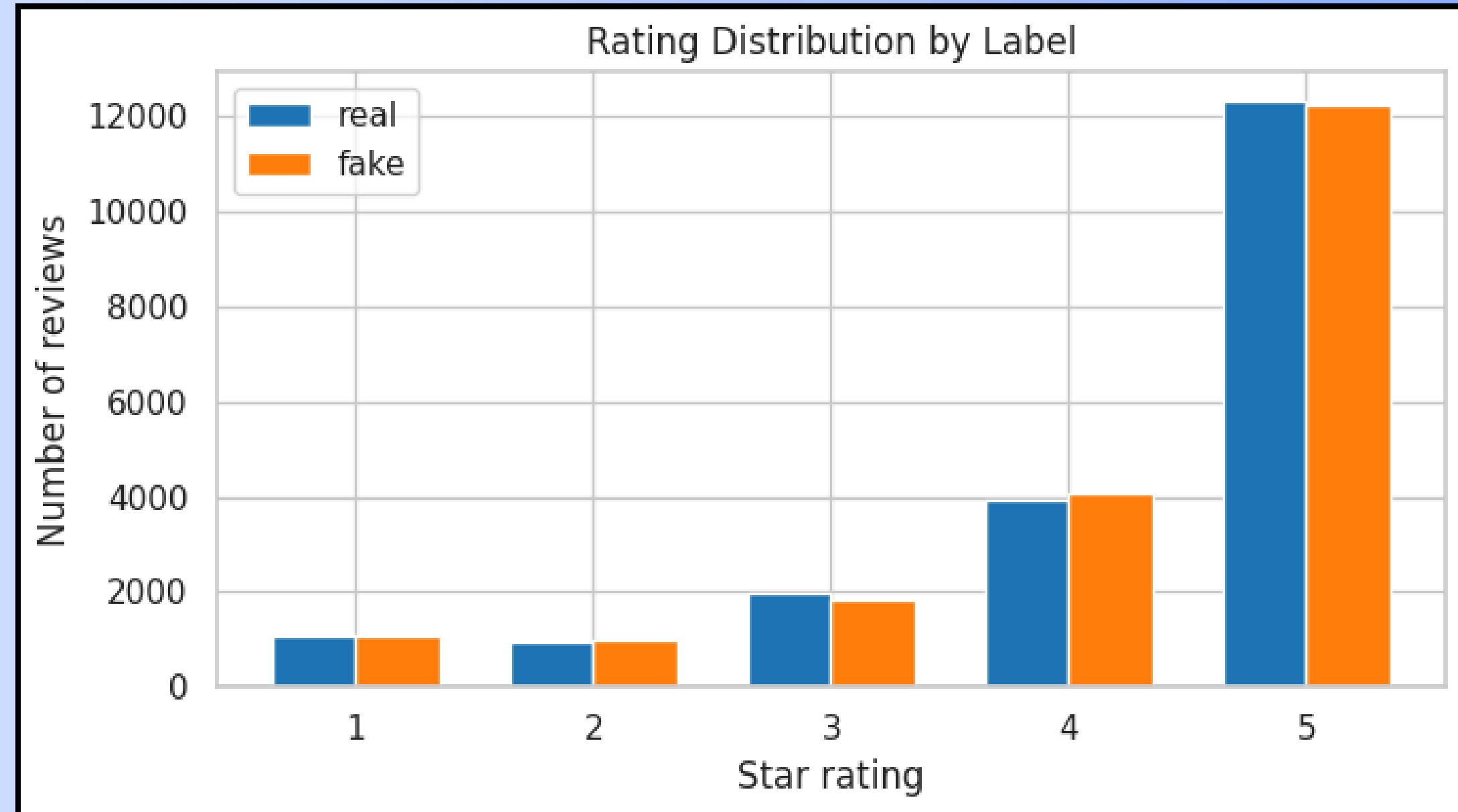
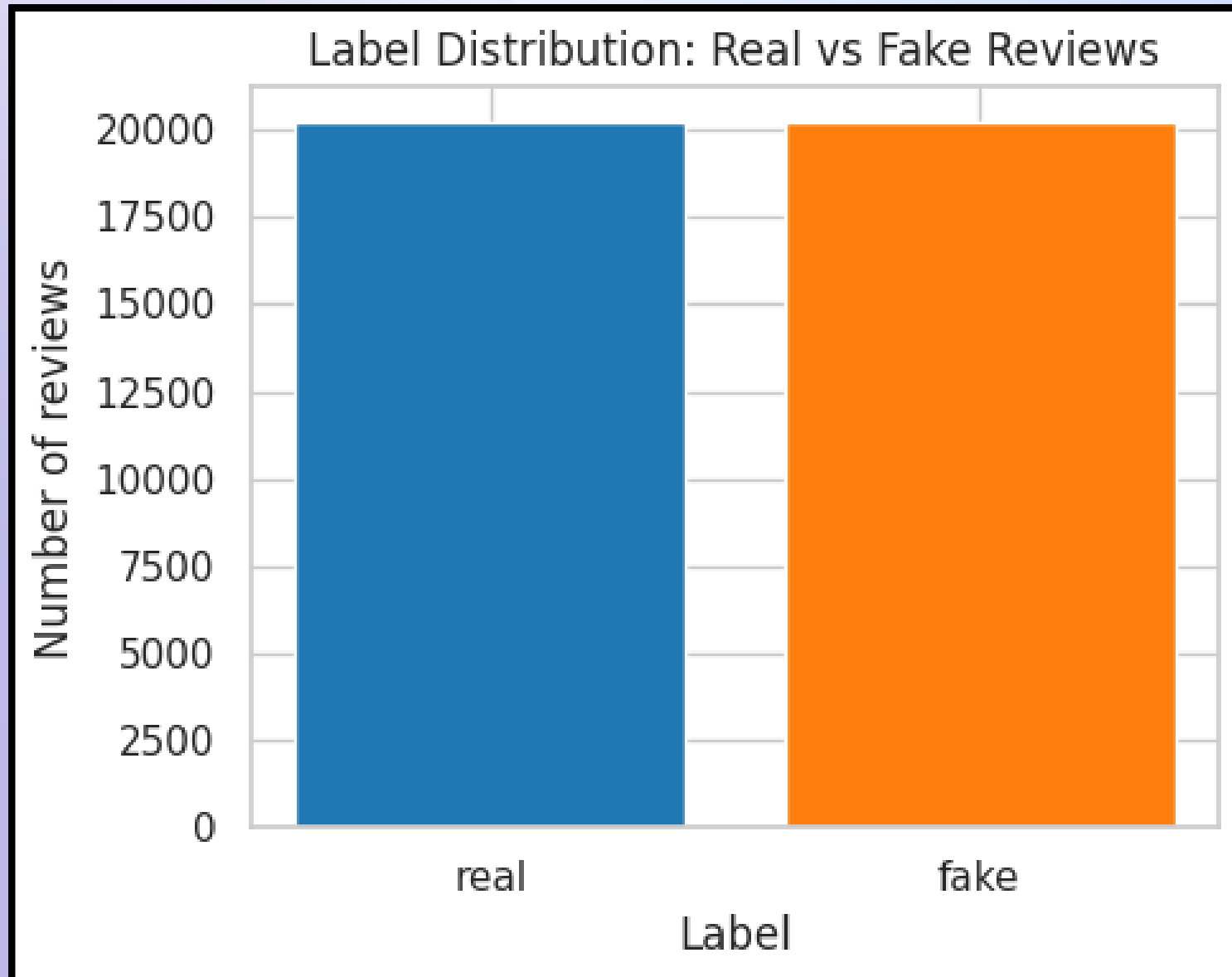
```
df["clean_text"] = df["text_"].apply(final_clean)

print("After final cleaning:", df.shape)
df[["text_", "clean_text"]].head()
```

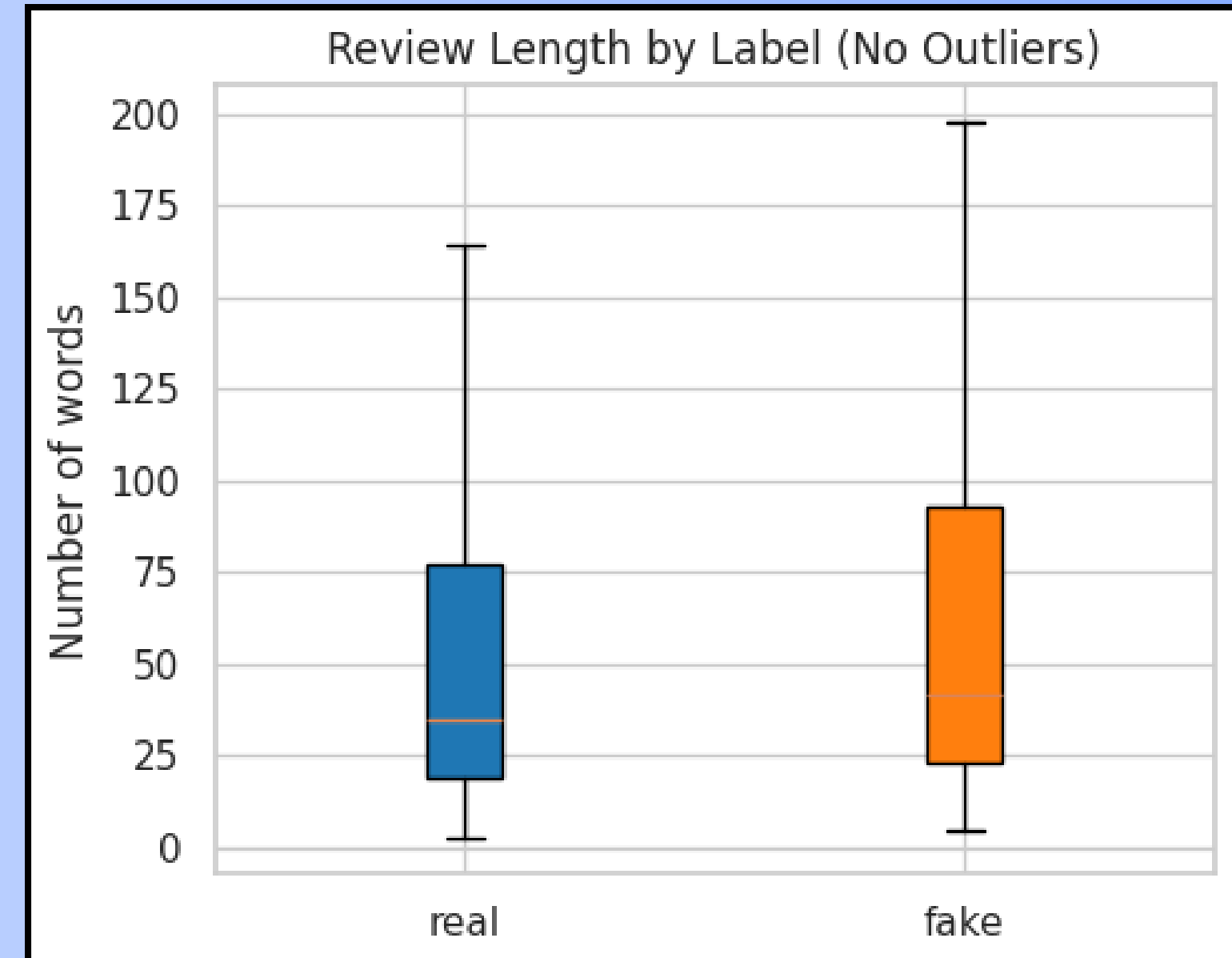
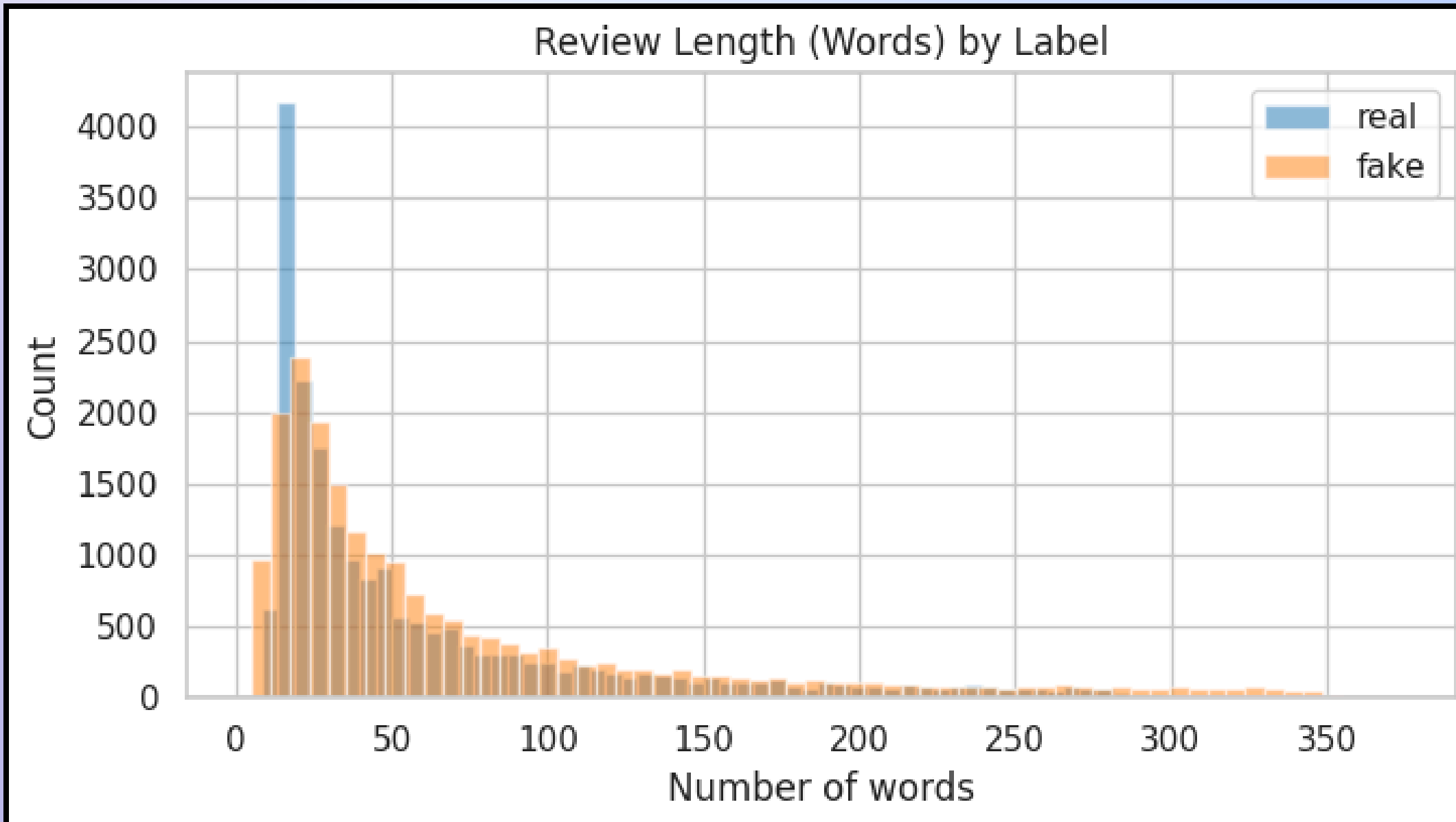
Cleaned Dataset Sample

category	rating	label_text	label_bin	word_len	char_len	clean_text
Home_and_Kitchen_5	5	real	0	12	75	love this well made sturdy and very comfortable...
Home_and_Kitchen_5	5	real	0	16	80	love it a great upgrade from the original ive...
Home_and_Kitchen_5	5	real	0	14	67	this pillow saved my back i love the look and...
Home_and_Kitchen_5	1	real	0	17	81	missing information on how to use it but it is...
Home_and_Kitchen_5	5	real	0	18	85	very nice set good quality we have had the set...
...	...	...	...	...	...	...
Home_and_Kitchen_5	2	fake	1	10	48	these are so weak they barely bring in any air
Home_and_Kitchen_5	5	fake	1	9	59	great set of glasses good quality comfortable...
Home_and_Kitchen_5	4	fake	1	10	45	it feels cool to the touch but is super hard
Home_and_Kitchen_5	5	fake	1	10	52	so cute made well my grand daughter will love...
Home_and_Kitchen_5	5	fake	1	10	62	works as expected for connecting cctv power su...

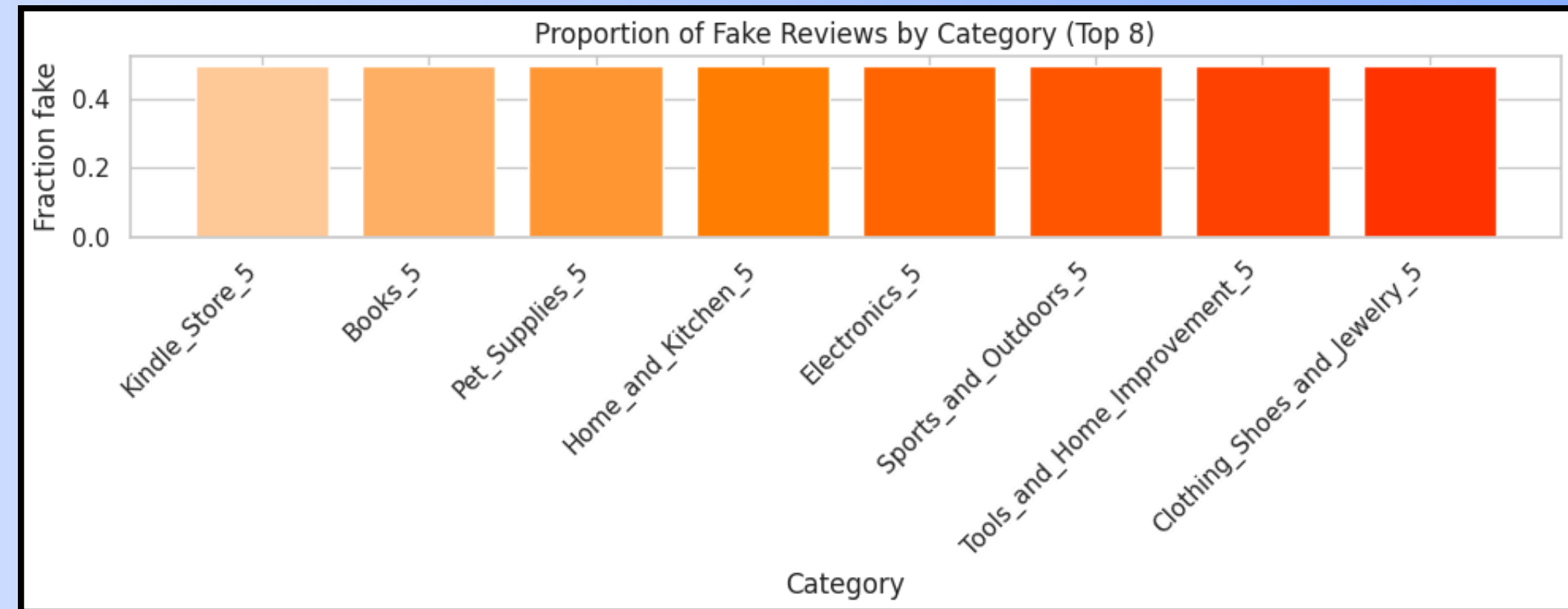
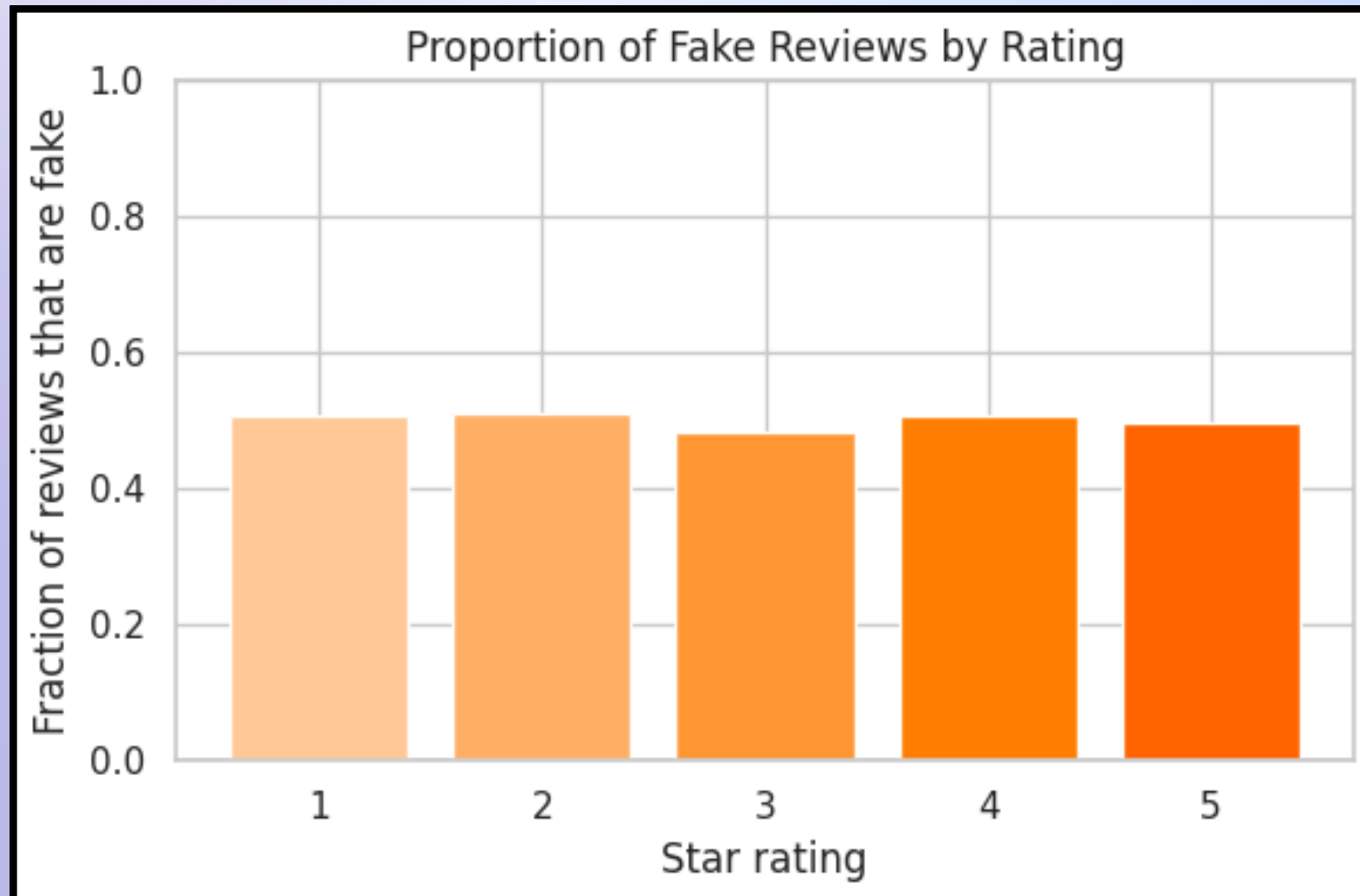
Data Visualization



Data Visualization



Data Visualization

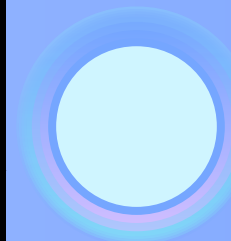




Knowledge Extracted

- Dataset has 40,432 reviews, balanced between real and fake, with no missing values.
- Fake reviews are generally shorter and simpler, while real reviews are longer and more detailed.
- Review lengths (words and characters) show clear differences between the two classes.
- Most reviews come from Home & Kitchen, giving a consistent domain.
- Cleaning removed noise and contradictions, resulting in high-quality text for modeling.
- These patterns provide strong signals that help ML models separate real vs fake reviews.

ML process steps:



ML Techniques Used	ML Terminologies
TF–IDF Vectorization (convert text → numeric features)	TF–IDF: word importance scoring method
Train–Test Split (80/20 stratified)	Feature Vector: numeric representation of text
Training Logistic Regression model	Label: 0 = real, 1 = fake
Training SVM model	Classification: predicting real vs fake
Evaluating models using accuracy, precision, recall, F1	Linear SVM: linear boundary classifier
Selecting best model (SVM)	Logistic Regression: probability-based classifier
Saving model + vectorizer for deployment	Evaluation Metrics: accuracy, precision, recall, F1

Train–Test Split and Feature Definition

X = clean_text (review text) y = label_bin (0 real, 1 fake)

Dataset: split into 80% training and 20% testing

```
from sklearn.model_selection import train_test_split

# Features (X) and labels (y)
X = df["clean_text"].values
y = df["label_bin"].values

# 80% training, 20% testing, preserving class balance
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Train size:", len(X_train))
print("Test size :", len(X_test))
```

TF-IDF Feature Extraction

We convert the cleaned text into numerical feature vectors using TF-IDF so the machine learning models can understand the text.

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Define TF-IDF vectorizer
tfidf = TfidfVectorizer(
    max_features=30000,
    ngram_range=(1, 2),    # unigrams + bigrams
    min_df=3               # ignore very rare terms
)

# Fit on training data only
tfidf.fit(X_train)

# Transform training and testing sets
X_train_tfidf = tfidf.transform(X_train)
X_test_tfidf   = tfidf.transform(X_test)

print("TF-IDF train shape:", X_train_tfidf.shape)
print("TF-IDF test shape :", X_test_tfidf.shape)
```


Logistic Regression Model

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score,
    recall_score, f1_score

# Define the Logistic Regression model
log_reg = LogisticRegression(
    max_iter=500,
    class_weight="balanced"
)

# Train the model
log_reg.fit(X_train_tfidf, y_train)

# Predict on the test set
y_pred_lr = log_reg.predict(X_test_tfidf)

# Evaluate performance
acc_lr = accuracy_score(y_test, y_pred_lr)
prec_lr = precision_score(y_test, y_pred_lr)
rec_lr = recall_score(y_test, y_pred_lr)
f1_lr = f1_score(y_test, y_pred_lr)

print("=== Logistic Regression ===")
print(f"Accuracy : {acc_lr:.4f}")
print(f"Precision: {prec_lr:.4f}")
print(f"Recall : {rec_lr:.4f}")
print(f"F1 Score : {f1_lr:.4f}")
```



SVM Model

```
from sklearn.svm import LinearSVC

# Define the Linear SVM model
svm_clf = LinearSVC()

# Train the model
svm_clf.fit(X_train_tfidf, y_train)

# Predict on the test set
y_pred_svm = svm_clf.predict(X_test_tfidf)

# Evaluate performance
acc_svm = accuracy_score(y_test, y_pred_svm)
prec_svm = precision_score(y_test, y_pred_svm)
rec_svm = recall_score(y_test, y_pred_svm)
f1_svm = f1_score(y_test, y_pred_svm)

print("=== Linear SVM ===")
print(f"Accuracy : {acc_svm:.4f}")
print(f"Precision: {prec_svm:.4f}")
print(f"Recall    : {rec_svm:.4f}")
print(f"F1 Score  : {f1_svm:.4f}")
```



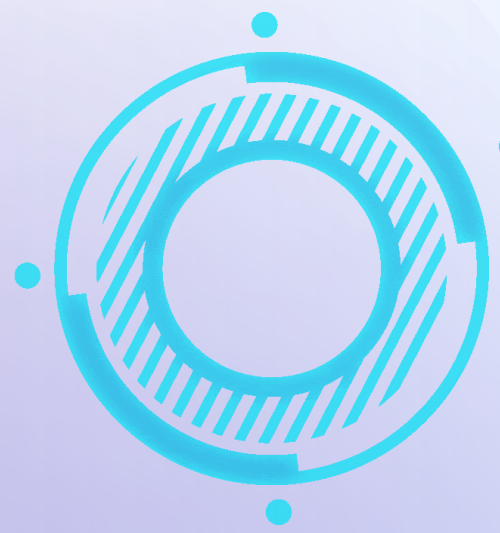
Model Selection and Saving

```
import joblib

# Choose the best-performing model (Linear SVM)
best_model = svm_clf

# Save the model and TF-IDF vectorizer
joblib.dump(best_model, "final_model.pkl")
joblib.dump(tfidf, "tfidf_vectorizer.pkl")

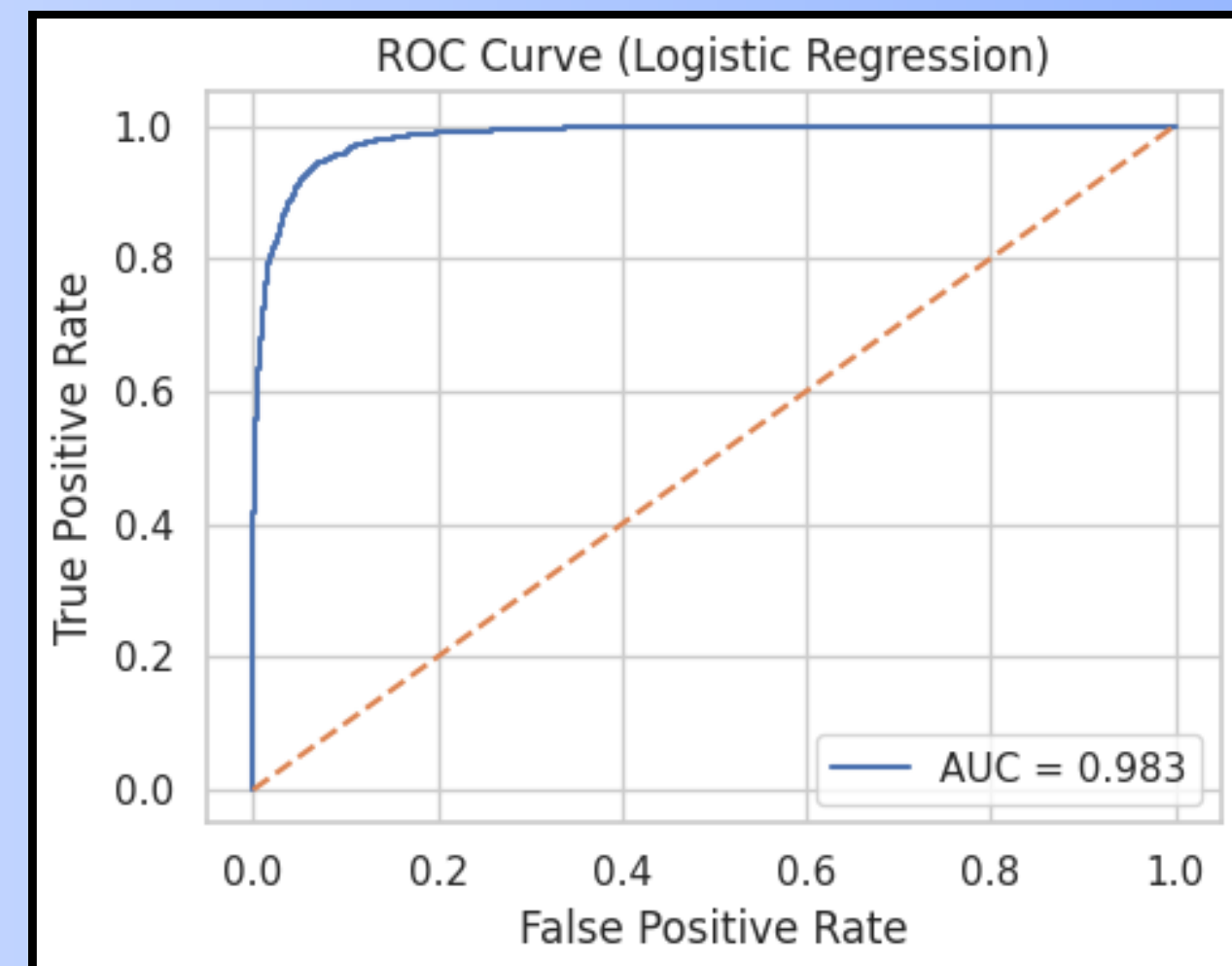
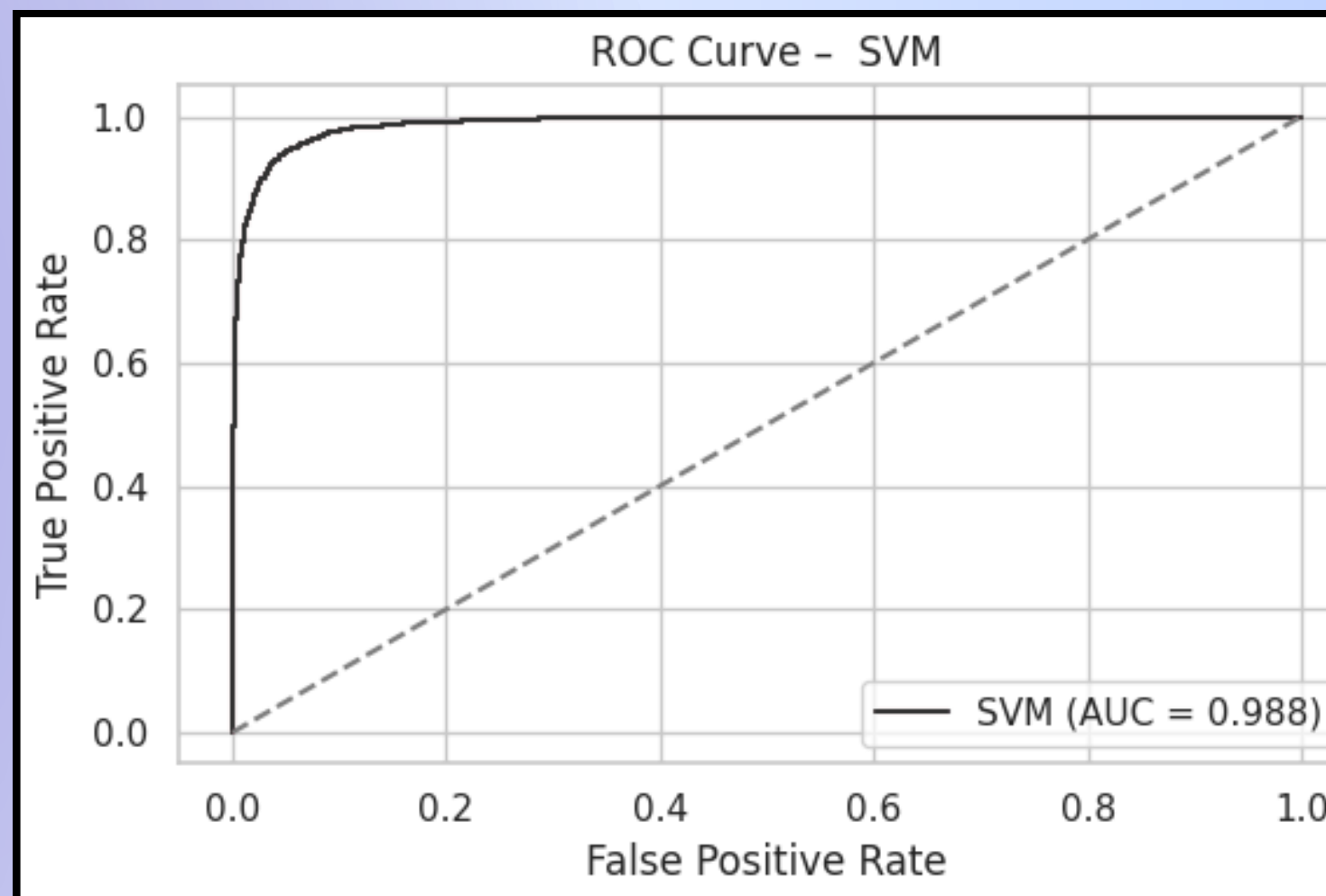
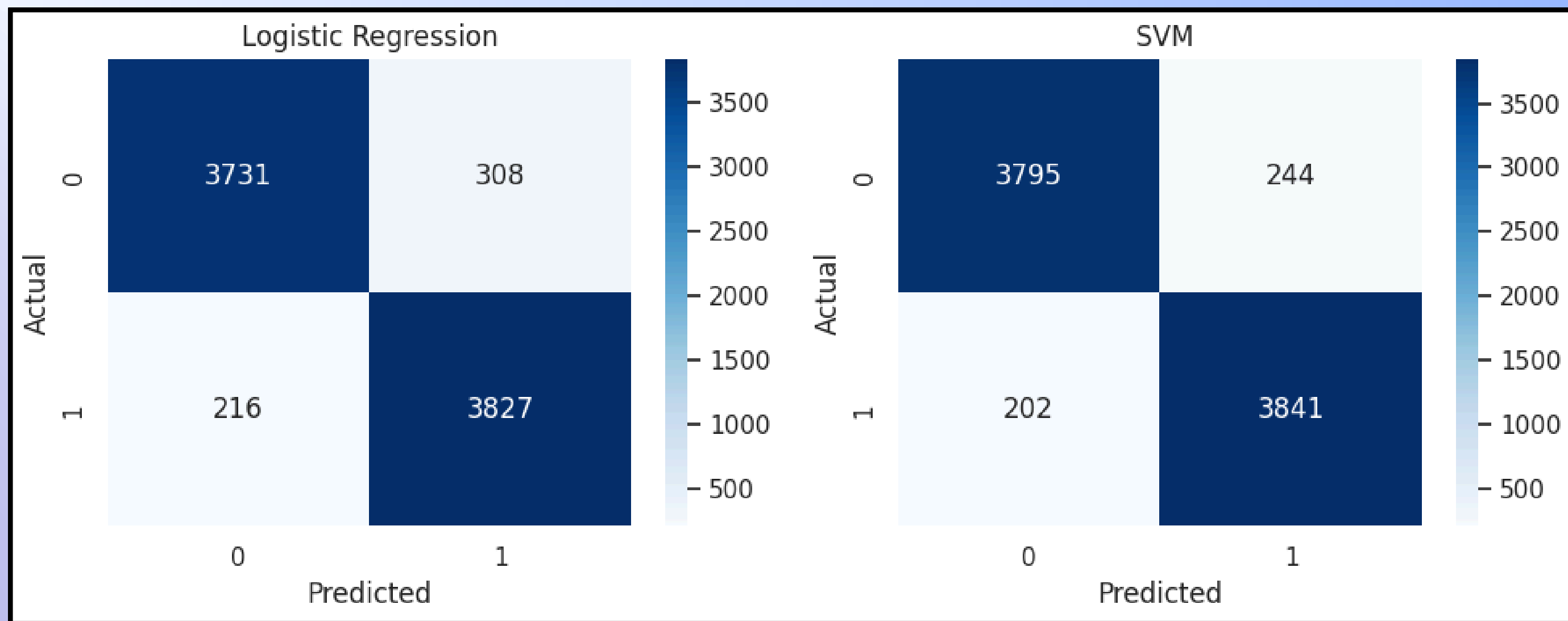
print("Saved: final_model.pkl and tfidf_vectorizer.pkl")
```



Model Results



Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.9352	0.9255	0.9466	0.9359
SVM	0.9448	0.9403	0.95	0.9451





Why SVM Performed Best

Handles high-dimensional text well

Strong margin-based classifier

Works great with TF-IDF

Best F1-score (0.9451)

Fake Review Detector 1.2.0 OAS 3.1

/openapi.json

default

GET	/healthz	Healthz	▼
GET	/metrics	Metrics	▼
POST	/predict	Predict	▼
POST	/reload	Reload Model	▼
POST	/auth/register	Auth Register	▼
POST	/auth/login	Auth Login	▼
GET	/auth/me	Auth Me	▼
POST	/auth/logout	Auth Logout	▼

Schemas

HTTPValidationError > Expand all object

ValidationError > Expand all object

Welcome back

Sign in to use the Fake Review Detector. No account yet? Register below.

Login

Register

Backend: FastAPI • SQLite • PBKDF2 • No user-defined classes • Frontend: Vanilla JS

Welcome back

Sign in to use the Fake Review Detector. No account yet? Register below.

Login

Register

Backend: FastAPI • SQLite • PBKDF2 • No user-defined classes • Frontend: Vanilla JS

Fake Review Detector

Paste a product review and get a prediction.

root@fake.com

Logout

Metrics

F1: 0.000

Precision: 0.000

Recall: 0.000

Accuracy: 0.500

Predict

Paste the review text here...

Predict

Backend: FastAPI • SQLite • PBKDF2 • No user-defined classes • Frontend: Vanilla JS

Fake Review Detector

root@fake.com

Logout

Paste a product review and get a prediction.

Metrics

F1: 0.000

Precision: 0.000

Recall: 0.000

Accuracy: 0.500

Predict

Works exactly as described.

Predict

Prediction: REAL (1)

Confidence: 56.9%

Fake Review Detector

Paste a product review and get a prediction.

root@fake.com

Logout

Metrics

F1: 0.000

Precision: 0.000

Recall: 0.000

Accuracy: 0.500

Predict

BEST PURCHASE EVER!!! Buy now!!!

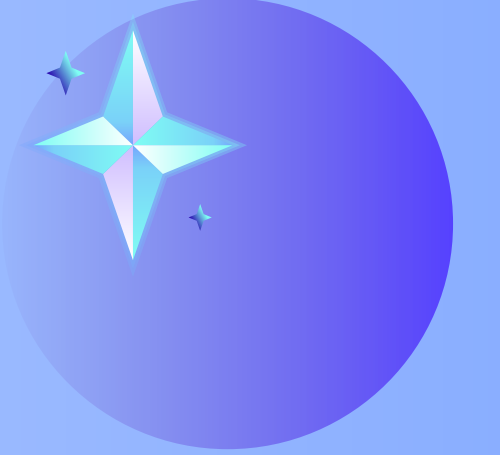
Predict

Prediction: **FAKE (0)**

Confidence: **48.1%**

Conlcosion

This project showed that machine learning can accurately detect fake reviews by using cleaned text data and TF-IDF features. After testing two models, SVM performed the best and was selected as the final classifier. The results highlight the importance of good preprocessing and feature extraction. A simple web interface was also created, allowing users to easily check whether a review is real or fake.



THANK YOU!